

Lecture 5: Decision Trees

by

Rajdip Nayek

Assistant Professor,
Applied Mechanics Department,
IIT Delhi

Instructor email: rajdipn@am.iitd.ac.in

Session agenda

01

Rules are powerful

How engineers already use sequential logic

02

Anatomy of a decision tree

Root, branches, leaves — and what they mean

03

How trees learn

Finding the best question to ask, without equations

04

Depth and overfitting

Why a too-precise tree can be dangerous

05

What can go wrong

Failure modes and how to spot them

Think of these questions

?

Will this equipment fail?

Noida Airport project:
classify compressor health
as NORMAL / MONITOR /
SHUTDOWN based on
vibration, temperature, and
run-hours.

!

**Will this project be
delayed?**

NYC infrastructure data:
predict whether a contract
will overrun its schedule
based on budget,
contractor grade, and
project type.

✓

Is this vendor high risk?

Score vendor bids on safety
record, past delays, and
resource availability —
producing a simple
APPROVE / FLAG / REJECT
decision.

What is a Decision Tree?

A decision tree **is a model that makes predictions by asking a series of YES/NO questions**, each question depending on the answer to the one before it.

IF-THEN rules

Every path through the tree is a rule:

"IF temp > 90 AND vibration > 5
THEN: fault likely"

Sequential decisions

Each question narrows down the answer.

Like a flowchart that thinks for itself.

Learns from data

The computer finds the BEST questions and thresholds automatically by studying past examples.

We already think in Decision Trees

Safety checklist

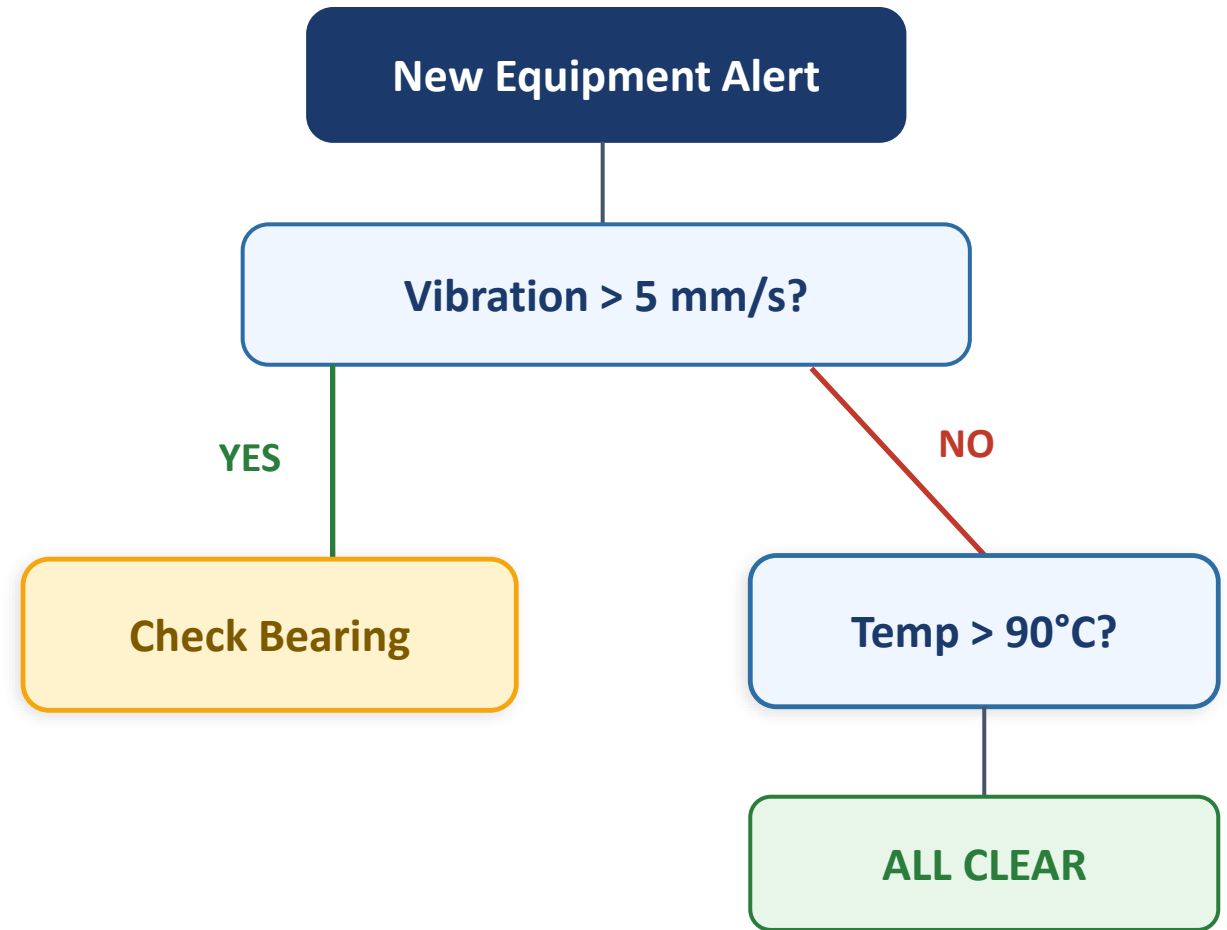
Is PPE worn? Is pressure within limit? Is area clear? — Classic IF-THEN.

Troubleshooting guide

"If vibration is high AND temperature is elevated, check bearing." That is a tree.

Bid approval

Is budget under threshold? Is contractor grade A? Is schedule achievable? APPROVE.



Two types of Decision Trees

Classification Tree

Predicts a CATEGORY

The answer is one of a fixed set of labels.

PASS / FAIL Weld inspection result

APPROVE / FLAG / REJECT Vendor risk assessment

ON TIME / DELAYED Project schedule prediction

NORMAL / MONITOR / FAULT Equipment health status

Regression Tree

Predicts a NUMBER

The answer is a continuous value — not a category.

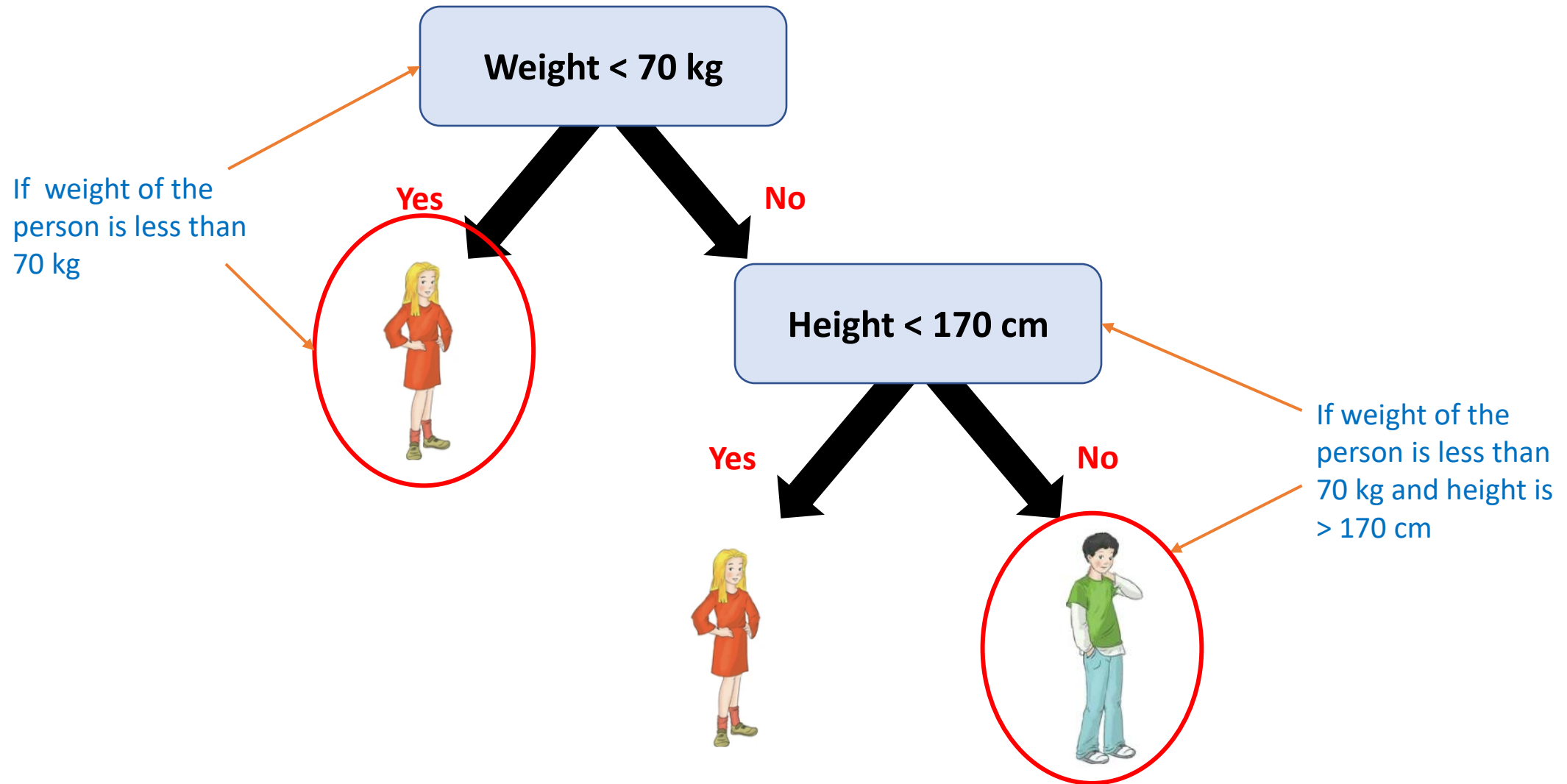
42 days Predicted schedule overrun

1.8cr Predicted cost overrun

87% Probability of equipment failure

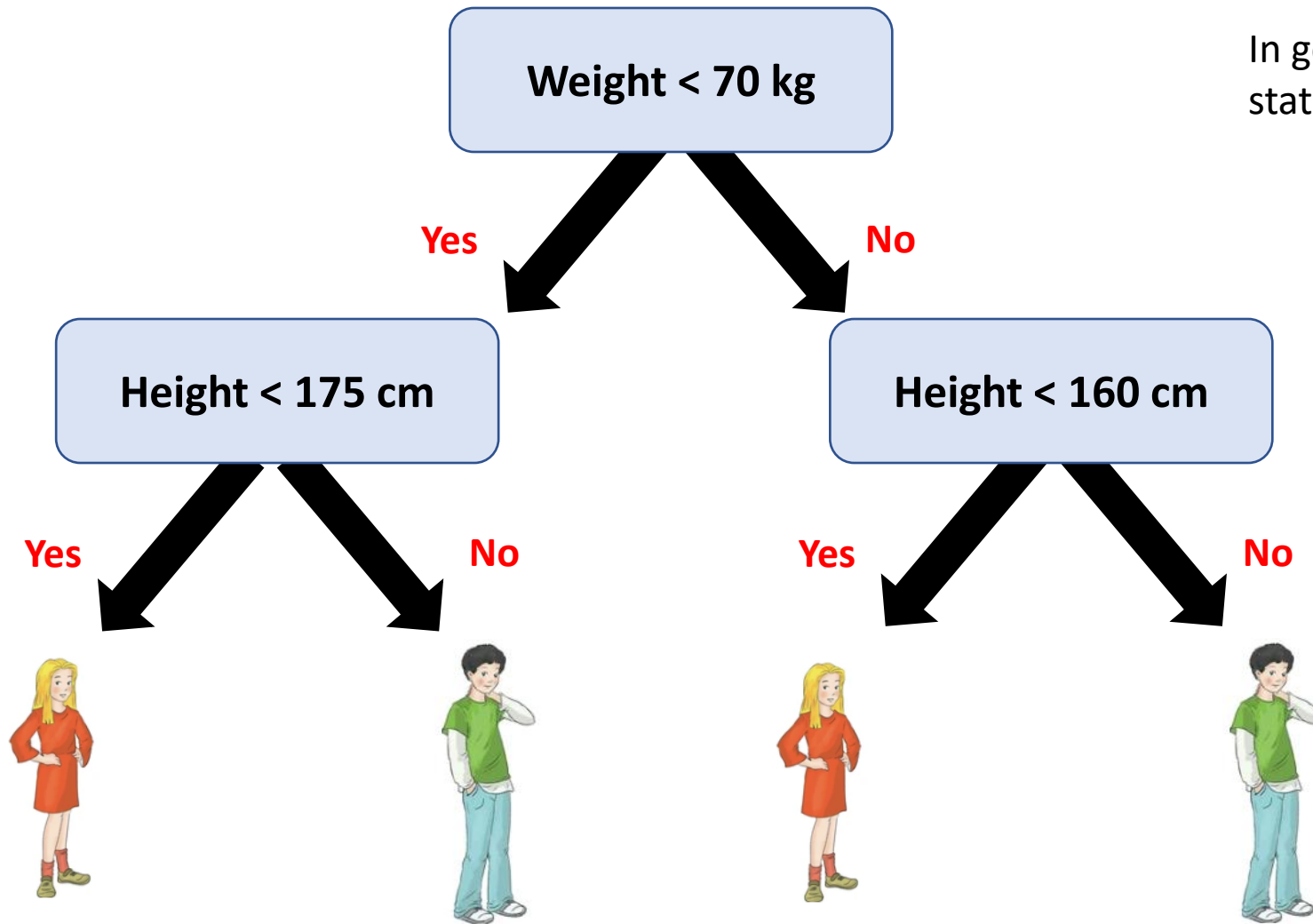
340 hours Remaining useful life of a pump

A simple decision tree based on rules



Split **numeric** input features by checking whether that feature is greater than or less than **some threshold**

Another possible decision tree

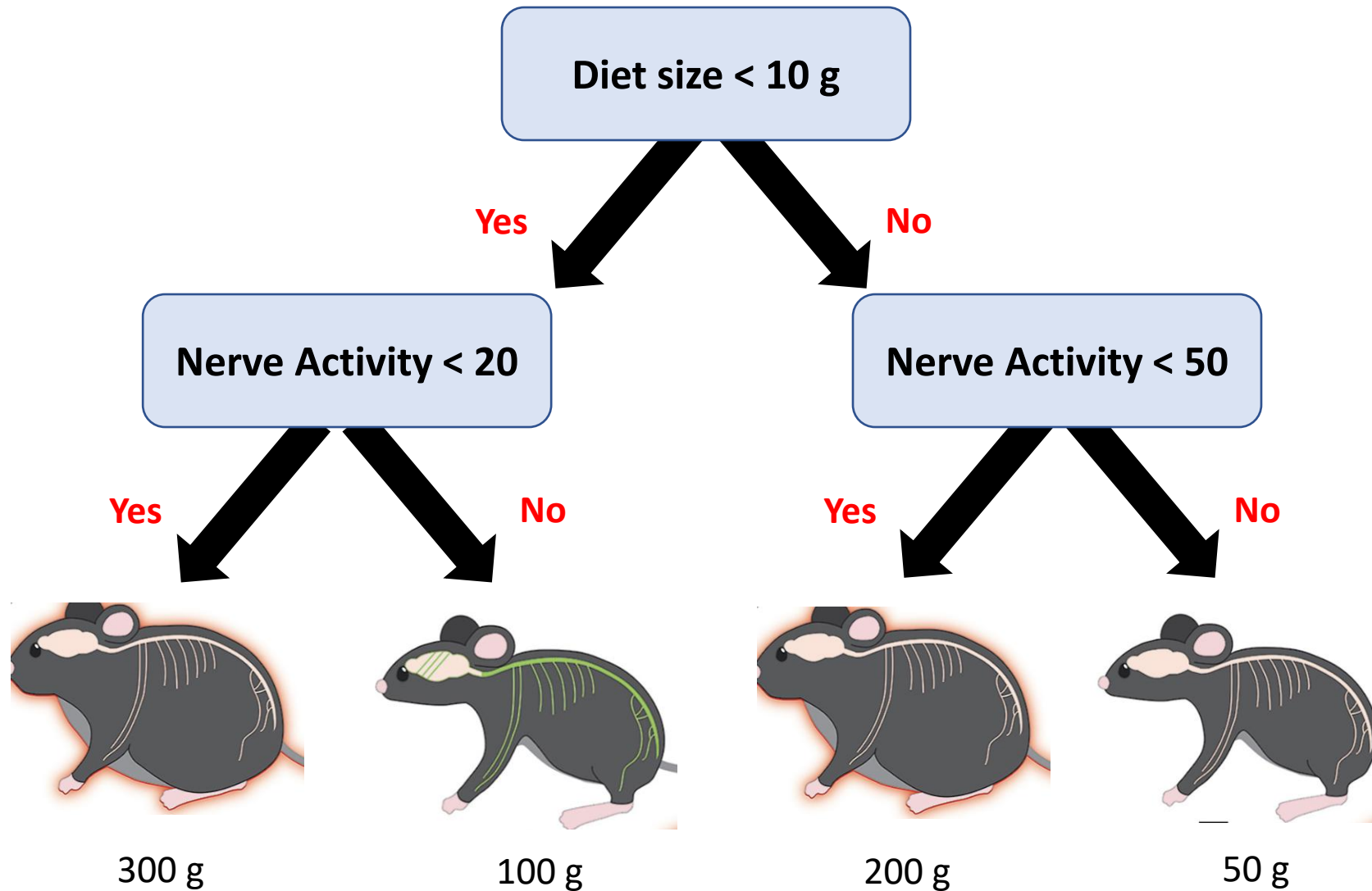


In general, a decision tree makes a statement...

... and then makes a decision based on whether or not the statement is true or false

Outputs are categorical → Classification problem → **Classification tree**

Regression tree



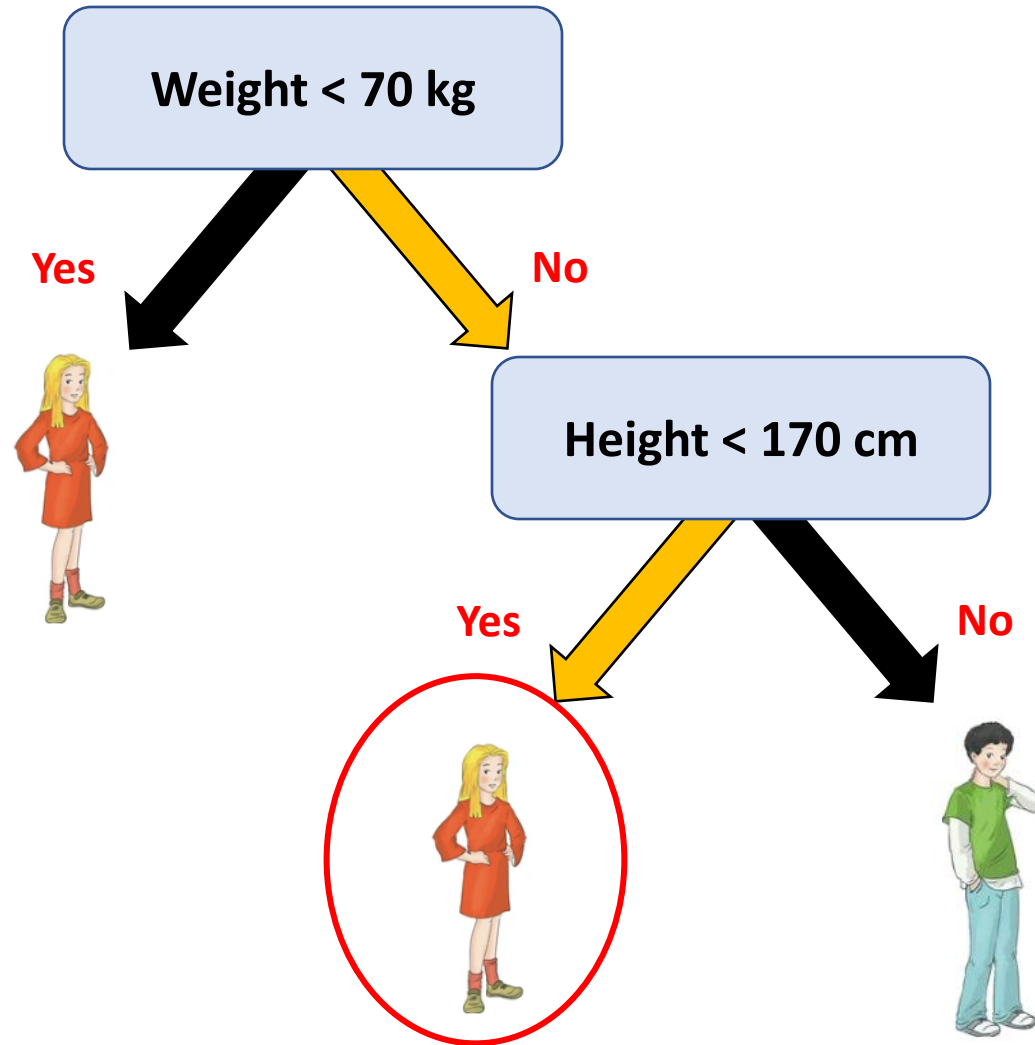
Outputs are numerical values → Regression problem → **Regression tree**

Prediction using decision trees (CART)



Test input

- Weight = 70 kg
- Height = 168 cm



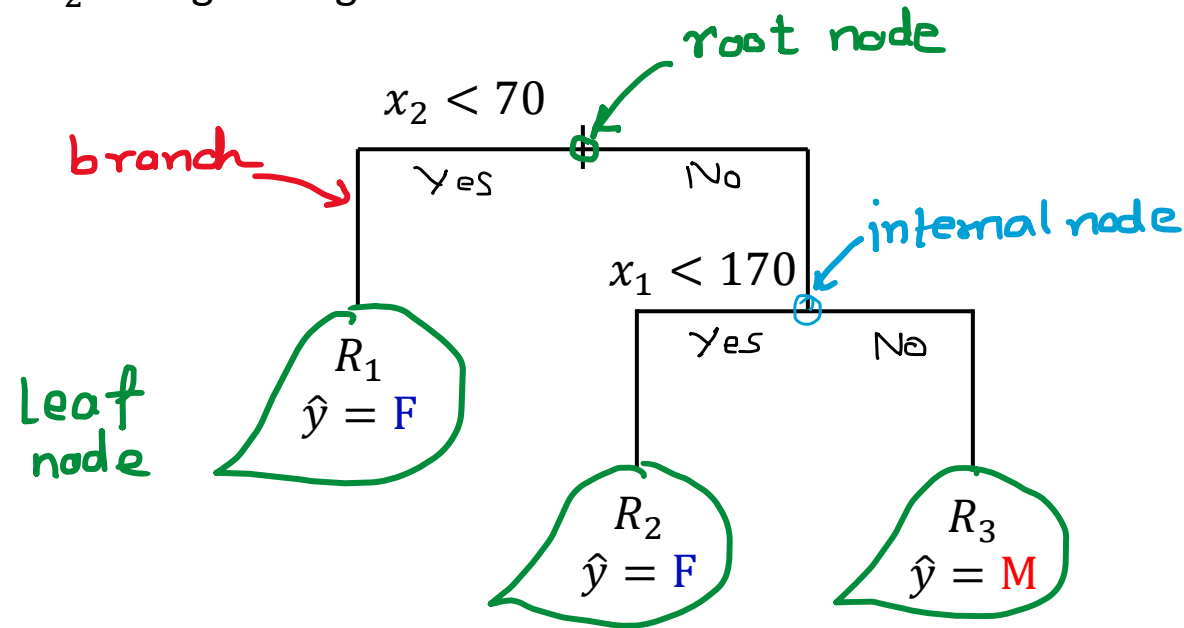
Make predictions by splitting on features according to a **tree structure**

Structure of a Decision Tree



Structure of a Decision Tree

- Consider the classification example with two numerical inputs $\mathbf{x} = [x_1 \ x_2]^T$ and a categorical output $y \in \{\text{F}, \text{M}\}$
 - x_1 - height in centimeters
 - x_2 - weight in kg



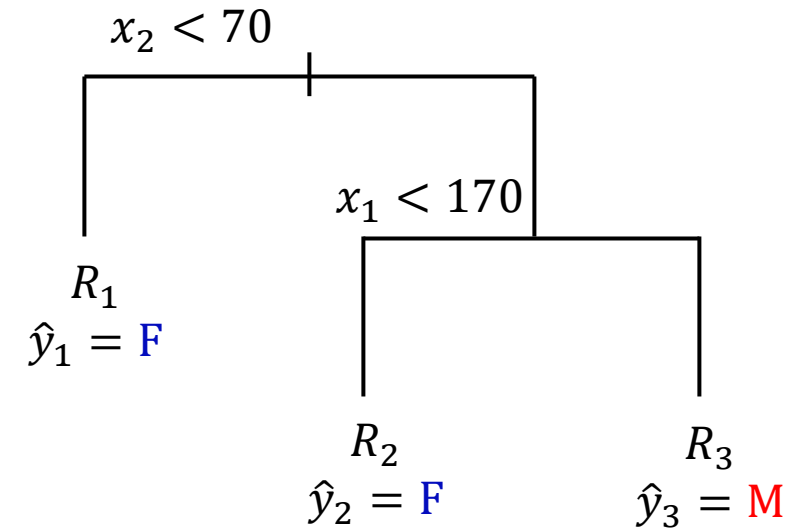
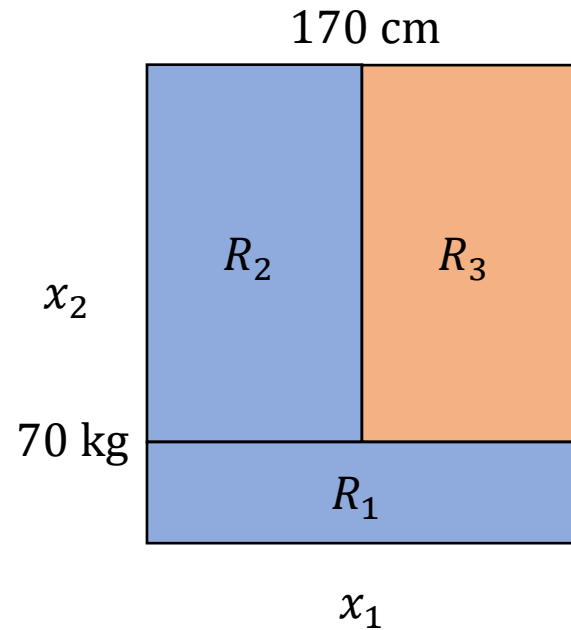
Structure of a **decision** tree

- Internal nodes (and root node) check an input variable for a condition
- Left/Right branch is determined by value of input variable $\rightarrow x_j < s_k$
- Leaf nodes are outputs (predictions)

The tree shown has two internal nodes (including the root) and three leaf nodes

Decision boundaries of a Decision Tree

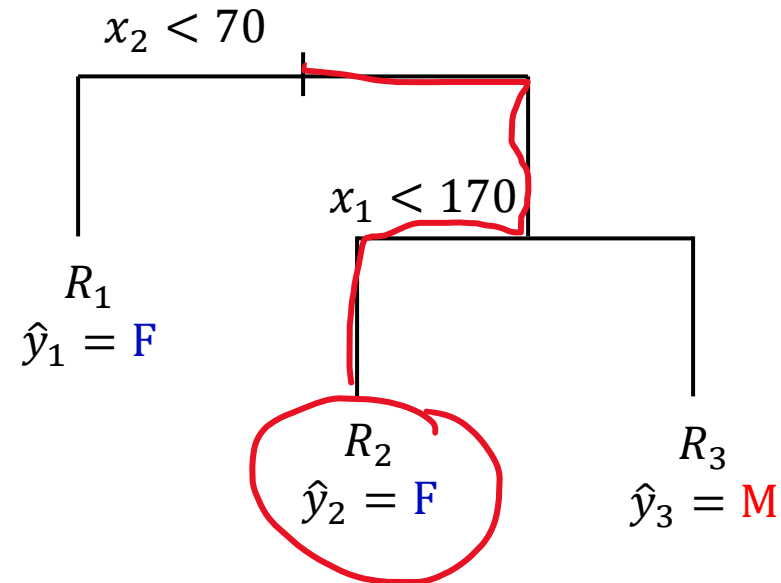
- Consider a classification example with two numerical inputs $\mathbf{x} = [x_1 \ x_2]^T$ and a categorical output $y \in \{\text{F}, \text{M}\}$
- Each path from **root node** to a **leaf** defines a **region** R_m of input space



- The decision tree partitions the input space into **axis-aligned 'boxes' or rectangles**. So the **decision boundaries** are in the shape of **rectangles**

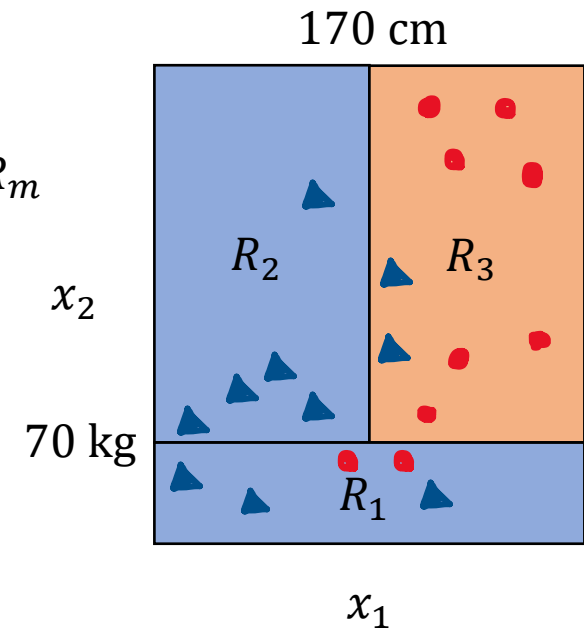
Make a prediction using a Classification tree

- Consider a test input $\mathbf{x}^* = [70 \quad 168]^T$
 - x_1 - height in centimeters
 - x_2 - weight in kg



Structure of Decision Trees

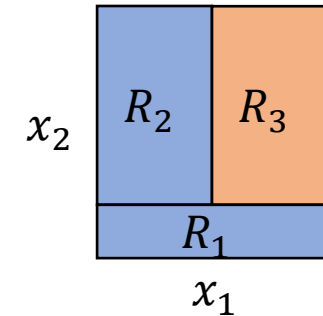
- The same partitioning can be done for a general input vector $\mathbf{x} = [x_1 \ x_2 \ \cdots \ x_p]^T$, however, the partitioned space is difficult to illustrate
- For a p -dimensional input space, the decision boundaries would be represented by **hyper-rectangles (boxes in higher dimensions)**
- Let $\{(\mathbf{x}_{m_1}, y_{m_1}), (\mathbf{x}_{m_2}, y_{m_2}), \dots, (\mathbf{x}_{m_k}, y_{m_k})\}$ be the training examples that fall into R_m
 - $m = 3, k = 9$ for the top-right reddish box
- **Regression tree**
 - Numerical output
 - Leaf value $\hat{y}_m$ typically set to the mean value in $\{(\mathbf{x}_{m_1}, y_{m_1}), \dots, (\mathbf{x}_{m_k}, y_{m_k})\}$
- **Classification tree**
 - Categorical output
 - Leaf value $\hat{y}_m$ typically set to the most common value (mode) in $\{(\mathbf{x}_{m_1}, y_{m_1}), \dots, (\mathbf{x}_{m_k}, y_{m_k})\}$



Learning Decision Trees

- We saw how a decision tree can be used to make a prediction. **Now, how a tree can be learned from training data?**

- Learning a decision tree involves deciding the **shape of the tree**
 - Finding the **number of regions (boxes)**, say L regions, $R_1, R_2, \dots, R_L$, and
 - Finding the **partitions of the boxes**



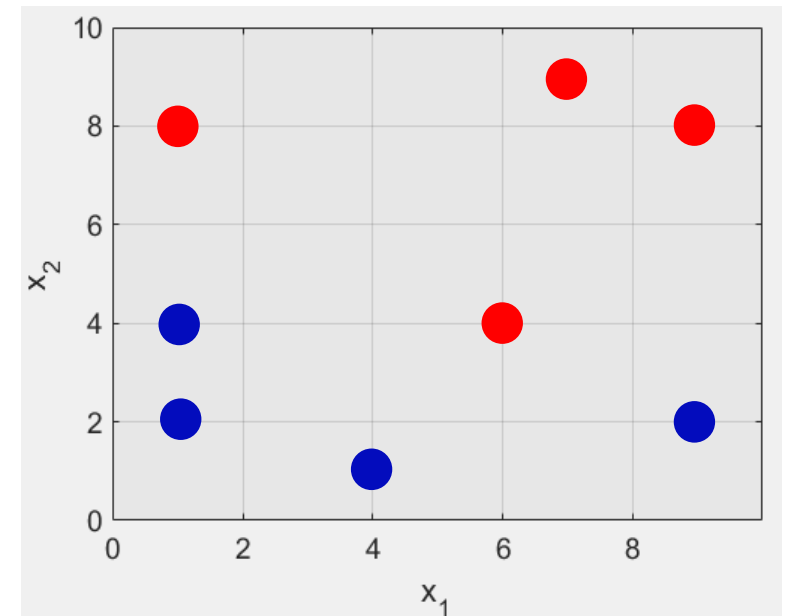
- We should select the number of regions and partitions such that the tree fits the training data well
 - the output predictions from the tree should match the output values in the training data
- However, finding the tree (a collection of splitting rules) that optimally partitions the input space to fit the training data is **computationally infeasible due to combinatorial explosion** in the number of ways you can partition the input space
- Searching through all possible binary trees is not possible in practice unless the tree size is very small
- To handle this situation, a heuristic algorithm known as **recursive binary splitting** is used for learning decision trees.

Learning **Classification** Trees using Recursive Binary Splitting

Start with the first split at the root and then build the tree from top to bottom

- When determining the splitting rule at the root node, the objective is to obtain a model that best explains the training data after a single split, without taking into consideration that additional splits may be added afterwards

i	x_1	x_2	y
1	9	2	Blue
2	4	1	Blue
3	1	2	Blue
4	1	4	Blue
5	1	8	Red
6	6	4	Red
7	7	9	Red
8	9	8	Red



Learning Classification Trees using Recursive Binary Splitting

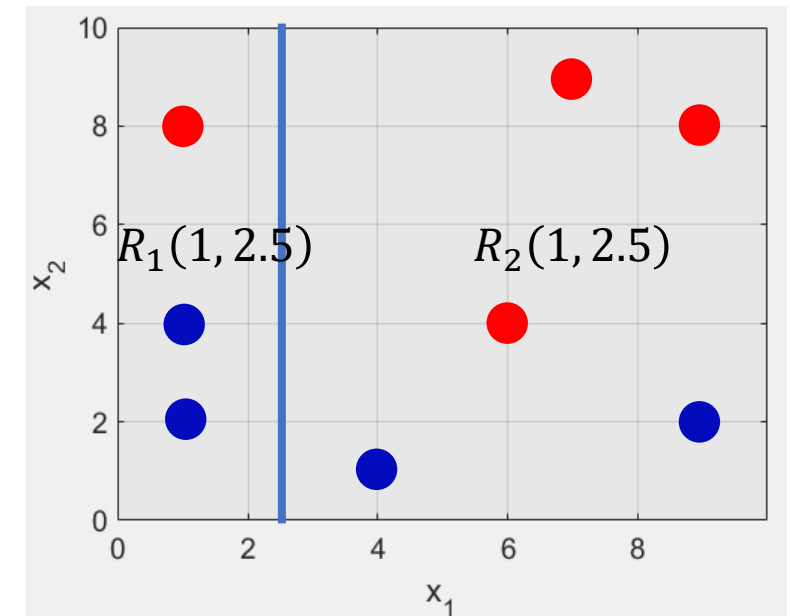
Start with the first split at the root and then build the tree from top to bottom

- When determining the splitting rule at the root node, the objective is to obtain a model that best explains the training data after a single split, without taking into consideration that additional splits may be afterwards
 - Select one of the p input variables $x_1, \dots, x_j, \dots, x_p$ and a corresponding cutpoint s which divide the input space into two half-spaces

$$R_1(j, s) = \{ \mathbf{x} \mid x_j < s \} \quad \text{and} \quad R_2(j, s) = \{ \mathbf{x} \mid x_j \geq s \}$$

i	x_1	x_2	y
1	9	2	Blue
2	4	1	Blue
3	1	2	Blue
4	1	4	Blue
5	1	8	Red
6	6	4	Red
7	7	9	Red
8	9	8	Red

Split at say $x_1 = 2.5$



Learning Classification Trees using Recursive Binary Splitting

Start with the first split at the root and then build the tree from top to bottom

- When determining the splitting rule at the root node, the objective is to obtain a model that best explains the training data after a single split, without taking into consideration that additional splits may be added afterwards
 - Select one of the p input variables $x_1, \dots, x_j, \dots, x_p$ and a corresponding cutpoint s which divide the input space into two half-spaces

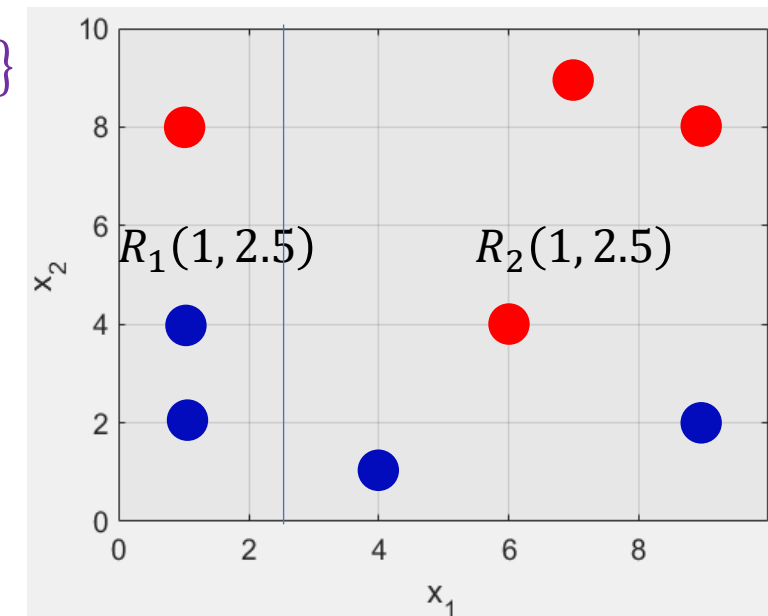
$$R_1(j, s) = \{ \mathbf{x} \mid x_j < s \} \quad \text{and} \quad R_2(j, s) = \{ \mathbf{x} \mid x_j \geq s \}$$

- The predictions associated with the two regions will be

$$\hat{y}_1(j, s) = \text{Mode}\{y^{(i)} : \mathbf{x}^{(i)} \in R_1(j, s)\} \quad \text{and} \quad \hat{y}_2(j, s) = \text{Mode}\{y^{(i)} : \mathbf{x}^{(i)} \in R_2(j, s)\}$$

Blue

Red

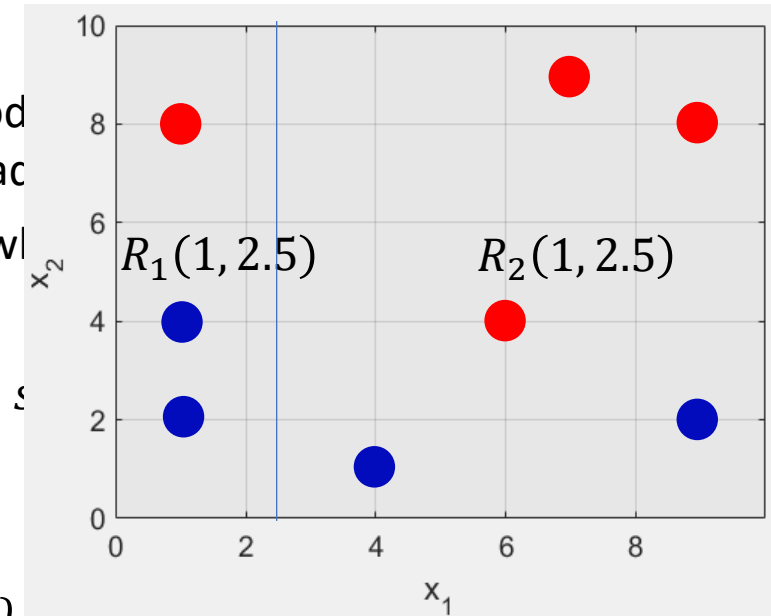


Learning Classification Trees using Recursive Binary Splitting

Start with the first split at the root and then build the tree from top to bottom

- When determining the splitting rule at the root node, the objective is to obtain a model that can predict the class of new data after a single split, without taking into consideration that additional splits may be needed
 - Select one of the p input variables $x_1, \dots, x_p$ and a corresponding cutpoint s with which to split the data into two half-spaces

$$R_1(j, s) = \{ \mathbf{x} \mid x_j < s \} \quad \text{and} \quad R_2(j, s) = \{ \mathbf{x} \mid x_j \geq s \}$$



- The predictions associated with the two regions will be

$$\hat{y}_1(j, s) = \text{Mode}\{y^{(i)} : \mathbf{x}^{(i)} \in R_1(j, s)\} \quad \text{and} \quad \hat{y}_2(j, s) = \text{Mode}\{y^{(i)} : \mathbf{x}^{(i)} \in R_2(j, s)\}$$

- Compute prediction loss for all training data points $(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^N$ at the node to determine goodness-of-fit

$$\text{Loss} = n_1 Q_{\ell, L}(j, s) + n_2 Q_{\ell, R}(j, s)$$

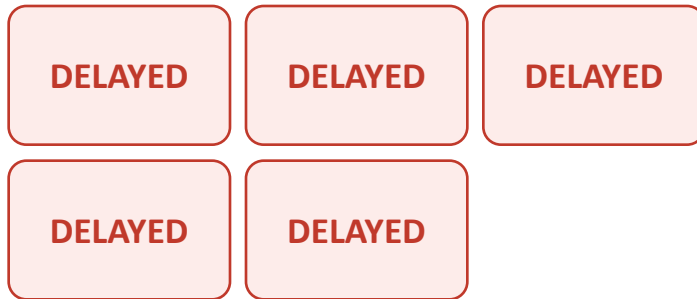
where, n_1, n_2 are the number of data points in left and right nodes of current split and $Q_{\ell, L}, Q_{\ell, R}$ are the associated prediction errors of the left (L) branch and right (R) branch for the ℓ th region

Loss → When learning a model, we use a **scalar number** to assess whether we are on track; **low is good**, high is bad

Learning **Classification** Trees using Recursive Binary Splitting

PERFECTLY PURE

Best possible outcome



Excellent split

FAIRLY PURE

Acceptable — 4:1 ratio



Good split

COMPLETELY MIXED

No information gained



Useless split

The algorithm scores every possible split and always chooses the one that creates the purest groups.

Prediction errors for **classification** trees

- Define $\hat{\pi}_{\ell,m}$ as the proportion of training observations in the ℓ th region belong to the m th class

$$\hat{\pi}_{\ell,m} = \frac{1}{n_{\ell}} \sum_{i: \mathbf{x}^{(i)} \in R_{\ell}(j,s)} \mathbf{I}(y^{(i)} = m)$$

Indicator function

- Misclassification rate:** proportion of data points in region R_{ℓ} which do not belong to the most common class

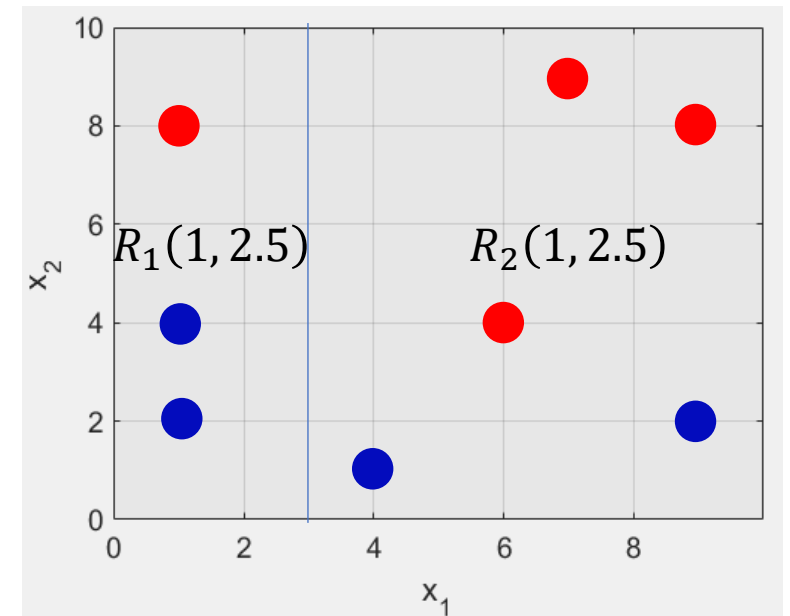
$$Q_{\ell} = 1 - \max_m \hat{\pi}_{\ell,m}$$

- Gini index**

$$Q_{\ell} = \sum_{m=1}^M \hat{\pi}_{\ell,m} (1 - \hat{\pi}_{\ell,m})$$

- Entropy criteria** (commonly used)

$$Q_{\ell} = - \sum_{m=1}^M \hat{\pi}_{\ell,m} \ln \hat{\pi}_{\ell,m}$$

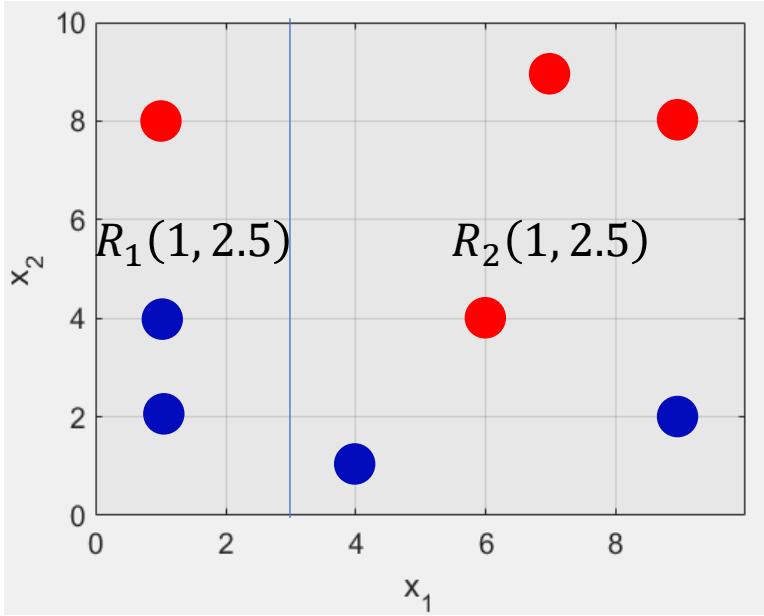


Calculating the loss for 1st split

- Define $\hat{\pi}_{\ell,m}$ as the proportion of training observations in the ℓ th region of that belong to the m th class

$$\hat{\pi}_{\ell,m} = \frac{1}{n_{\ell}} \sum_{i:\mathbf{x}^{(i)} \in R_{\ell}(j,s)} \mathbf{I}(y^{(i)} = m)$$

- Let's use **Misclassification rate**: $Q_{\ell} = 1 - \max_m \hat{\pi}_{\ell,m}$



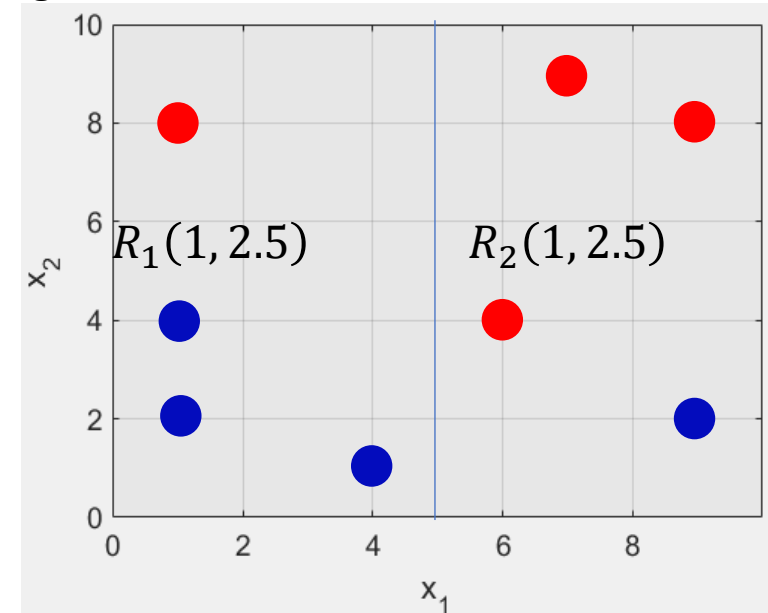
Splits (R_1)	n_1	$\hat{\pi}_{1,B}$	$\hat{\pi}_{1,R}$	Q_1	n_2	$\hat{\pi}_{2,B}$	$\hat{\pi}_{2,R}$	Q_2	$n_1 Q_1 + n_2 Q_2$
$x_1 < 2.5$	3	2/3	1/3	1/3	5	2/5	3/5	2/5	3

Calculating the loss for 1st split

- Define $\hat{\pi}_{\ell,m}$ as the proportion of training observations in the ℓ th region of that belong to the m th class

$$\hat{\pi}_{\ell,m} = \frac{1}{n_{\ell}} \sum_{i: \mathbf{x}^{(i)} \in R_{\ell}(j,s)} \mathbf{I}(y^{(i)} = m)$$

- Let's use **Misclassification rate**: $Q_{\ell} = 1 - \max_m \hat{\pi}_{\ell,m}$



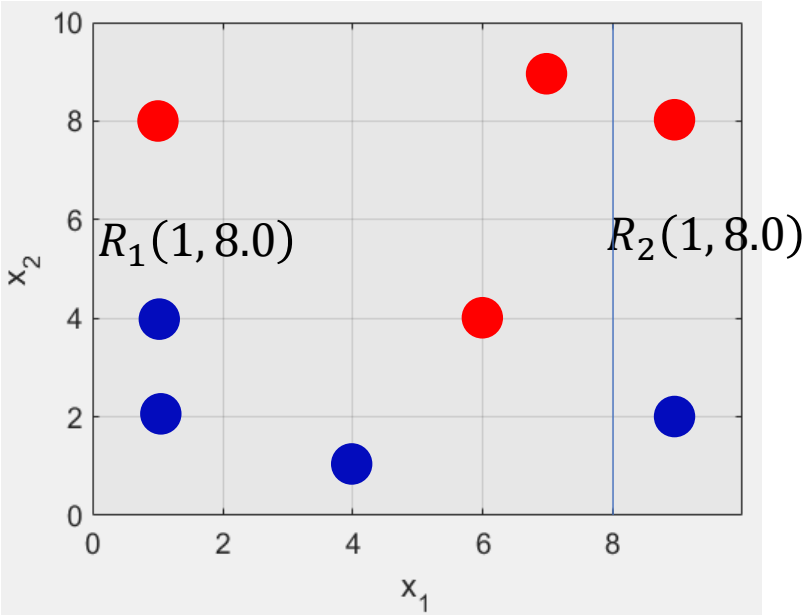
Splits (R_1)	n_1	$\hat{\pi}_{1,B}$	$\hat{\pi}_{1,R}$	Q_1	n_2	$\hat{\pi}_{2,B}$	$\hat{\pi}_{2,R}$	Q_2	$n_1 Q_1 + n_2 Q_2$
$x_1 < 2.5$	3	2/3	1/3	1/3	5	2/5	3/5	2/5	3
$x_1 < 5.0$	4	3/4	1/4	1/4	4	1/4	3/4	1/4	2

Calculating the loss for 1st split

- Define $\hat{\pi}_{\ell,m}$ as the proportion of training observations in the ℓ th region of that belong to the m th class

$$\hat{\pi}_{\ell,m} = \frac{1}{n_{\ell}} \sum_{i:\mathbf{x}^{(i)} \in R_{\ell}(j,s)} \mathbf{I}(y^{(i)} = m)$$

- Let's use **Misclassification rate**: $Q_{\ell} = 1 - \max_m \hat{\pi}_{\ell,m}$



Splits (R_1)	n_1	$\hat{\pi}_{1,B}$	$\hat{\pi}_{1,R}$	Q_1	n_2	$\hat{\pi}_{2,B}$	$\hat{\pi}_{2,R}$	Q_2	$n_1Q_1 + n_2Q_2$
$x_1 < 2.5$	3	2/3	1/3	1/3	5	2/5	3/5	2/5	3
$x_1 < 5.0$	4	3/4	1/4	1/4	4	1/4	3/4	1/4	2
$x_1 < 8.0$	6	3/6	3/6	3/6	2	1/2	1/2	1/2	4

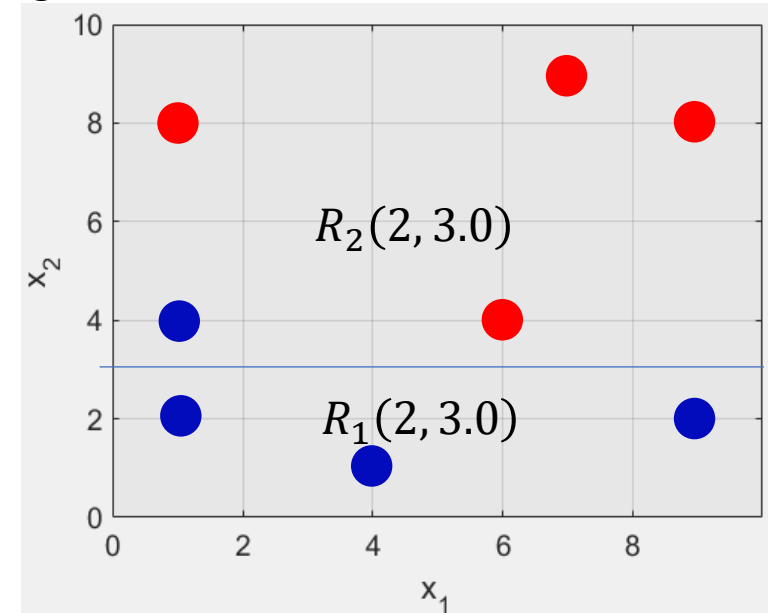
Calculating the loss for 1st split

- Define $\hat{\pi}_{\ell,m}$ as the proportion of training observations in the ℓ th region of that belong to the m th class

$$\hat{\pi}_{\ell,m} = \frac{1}{n_{\ell}} \sum_{i: \mathbf{x}^{(i)} \in R_{\ell}(j,s)} \mathbf{I}(y^{(i)} = m)$$

- Let's use **Misclassification rate**: $Q_{\ell} = 1 - \max_m \hat{\pi}_{\ell,m}$

- To find the optimal split, we select the values for j and s that minimise the loss



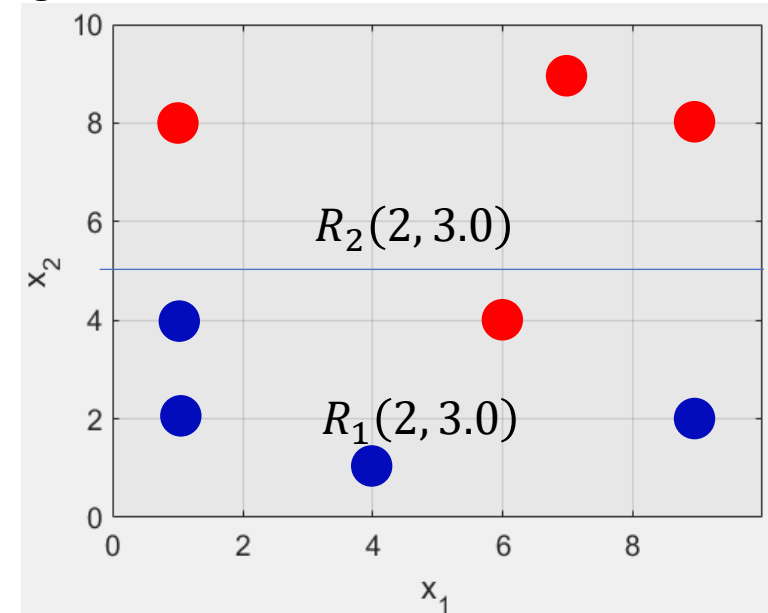
Splits (R_1)	n_1	$\hat{\pi}_{1,B}$	$\hat{\pi}_{1,R}$	Q_1	n_2	$\hat{\pi}_{2,B}$	$\hat{\pi}_{2,R}$	Q_2	$n_1 Q_1 + n_2 Q_2$
$x_1 < 2.5$	3	2/3	1/3	1/3	5	2/5	3/5	2/5	3
$x_1 < 5.0$	4	3/4	1/4	1/4	4	1/4	3/4	1/4	2
$x_1 < 8.0$	6	3/6	3/6	3/6	2	1/2	1/2	1/2	4
$x_2 < 3.0$	3	3/3	0/3	0/3	5	1/5	4/5	1/5	1

Calculating the loss for 1st split

- Define $\hat{\pi}_{\ell,m}$ as the proportion of training observations in the ℓ th region of that belong to the m th class

$$\hat{\pi}_{\ell,m} = \frac{1}{n_{\ell}} \sum_{i: \mathbf{x}^{(i)} \in R_{\ell}(j,s)} \mathbf{I}(y^{(i)} = m)$$

- Let's use **Misclassification rate**: $Q_{\ell} = 1 - \max_m \hat{\pi}_{\ell,m}$



Splits (R_1)	n_1	$\hat{\pi}_{1,B}$	$\hat{\pi}_{1,R}$	Q_1	n_2	$\hat{\pi}_{2,B}$	$\hat{\pi}_{2,R}$	Q_2	$n_1 Q_1 + n_2 Q_2$
$x_1 < 2.5$	3	2/3	1/3	1/3	5	2/5	3/5	2/5	3
$x_1 < 5.0$	4	3/4	1/4	1/4	4	1/4	3/4	1/4	2
$x_1 < 8.0$	6	3/6	3/6	3/6	2	1/2	1/2	1/2	4
$x_2 < 3.0$	3	3/3	0/3	0/3	5	1/5	4/5	1/5	1
$x_2 < 5.0$	5	4/5	1/5	1/5	3	0/3	3/3	0/3	1

Learning Classification Trees using Recursive Binary Splitting

Start with the first split at the root and then build the tree from top to bottom

- When determining the splitting rule at the root node, the objective is to obtain a model that best explains the training data after a single split, without taking into consideration that additional splits may be added before arriving at the final model
 - Select one of the p input variables $x_1, \dots, x_p$ and a corresponding cutpoint s which divide the input space into two half-spaces

$$R_1(j, s) = \{ \mathbf{x} \mid x_j < s \} \text{ and } R_2(j, s) = \{ \mathbf{x} \mid x_j \geq s \}$$

- The predictions associated with the two regions will be

$$\hat{y}_1(j, s) = \text{Mode}\{y^{(i)} : \mathbf{x}^{(i)} \in R_1(j, s)\} \text{ and } \hat{y}_2(j, s) = \text{Mode}\{y^{(i)} : \mathbf{x}^{(i)} \in R_2(j, s)\}$$

- Compute *loss* (squared error) for all training data points $(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^N$ at the node to determine goodness-of-fit

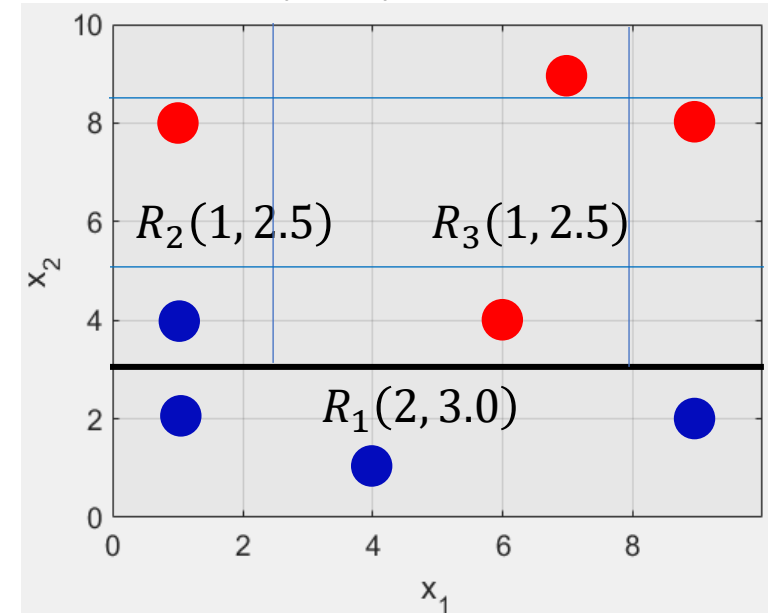
$$n_1 Q_1 + n_2 Q_2$$

where, n_1, n_2 are # of data points in left and right nodes of current split and Q_1, Q_2 are associated prediction errors

- To find the optimal split, we select the values for j and s that minimise the loss
- When we have decided on the first split of the input space (corresponding to the root node of the tree), this split is kept fixed, and we continue in a similar way for the two resulting half-spaces (corresponding to the two branches of the tree)

Calculating the loss for 2nd split

- There is no point splitting region R_1 further since it only contains data points from the same class (Blue)
- We therefore split the upper region into two new regions, R_2 and R_3
- We split in the same manner as before



Splits (R_2)	n_2	$\hat{\pi}_{2,B}$	$\hat{\pi}_{2,R}$	Q_2	n_3	$\hat{\pi}_{3,B}$	$\hat{\pi}_{3,R}$	Q_3	$n_2Q_2 + n_3Q_3$
$x_1 < 2.5$	2	1/2	1/2	1/2	3	0/3	3/3	0/3	1
$x_1 < 8.0$	4	1/4	3/4	1/4	1	0/1	1/1	0/1	1
$x_2 < 5.0$	2	1/2	1/2	1/2	3	0/3	3/3	0/3	1
$x_2 < 8.5$	4	1/4	3/4	1/4	1	0/1	1/1	0/1	1

Learning Regression Trees using Recursive Binary Splitting

Start with the first split at the root and then build the tree from top to bottom

- When determining the splitting rule at the root node, the objective is to obtain a model that best explains the training data after a single split, without taking into consideration that additional splits may be added before arriving at the final model
 - Select one of the p input variables $x_1, \dots, x_p$ and a corresponding cutpoint s which divide the input space into two half-spaces

$$R_1(j, s) = \{ \mathbf{x} \mid x_j < s \} \text{ and } R_2(j, s) = \{ \mathbf{x} \mid x_j \geq s \}$$

- The predictions associated with the two regions will be

$$\hat{y}_1(j, s) = \text{Mean}\{y^{(i)} : \mathbf{x}^{(i)} \in R_1(j, s)\} \text{ and } \hat{y}_2(j, s) = \text{Mean}\{y^{(i)} : \mathbf{x}^{(i)} \in R_2(j, s)\}$$

- Compute *loss* (squared error) for all training data points $(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^N$ at the node to determine goodness-of-fit

$$n_1 Q_1 + n_2 Q_2 = \sum_{i: \mathbf{x}^{(i)} \in R_1(j, s)} (y^{(i)} - \hat{y}_1(j, s))^2 + \sum_{i: \mathbf{x}^{(i)} \in R_2(j, s)} (y^{(i)} - \hat{y}_2(j, s))^2$$

where, n_1, n_2 are # of data points in left and right nodes of current split and Q_1, Q_2 are associated prediction errors

- To find the optimal split, we select the values for j and s that minimise the loss
- When we have decided on the first split of the input space (corresponding to the root node of the tree), this split is kept fixed, and we continue in a similar way for the two resulting half-spaces (corresponding to the two branches of the tree)

Algorithm for DT using Recursive Binary Splitting

Data: $\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$

Output: Decision tree with regions $R_1, R_2, \dots, R_L$ and corresponding predictions $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_L$

1. Let R denote the entire input space
2. Compute the regions $(R_1, \dots, R_L) = \mathbf{split}(R, \mathcal{T})$
3. Compute the predictions $\hat{y}_\ell$ for $\ell = 1, 2, \dots, L$ as

$$\hat{y}_\ell = \begin{cases} \text{Mean}\{y^{(i)} : \mathbf{x}^{(i)} \in R_\ell\} & \text{(Regression)} \\ \text{Mode}\{y^{(i)} : \mathbf{x}^{(i)} \in R_\ell\} & \text{(Classification)} \end{cases}$$

Test Data: $\mathbf{x}^*$

Output: $\hat{y}(\mathbf{x}^*)$

1. Find the region R_ℓ in which $\mathbf{x}^*$ belongs to
2. Return the prediction $\hat{y}(\mathbf{x}^*) = \hat{y}_\ell$

function **split**($R_\ell, \mathcal{T}_\ell$)

if stopping criterion fulfilled

return R_ℓ

else

Go through all possible splits $x_j < s$ for all input variables

Pick pair (j, s) that minimizes the chosen loss

Split the region R_ℓ into two half-spaces R_ℓ and $R_{\ell+1}$

Split the data $\mathcal{T}_\ell$ into two parts, $\mathcal{T}_\ell$ and $\mathcal{T}_{\ell+1}$, respectively

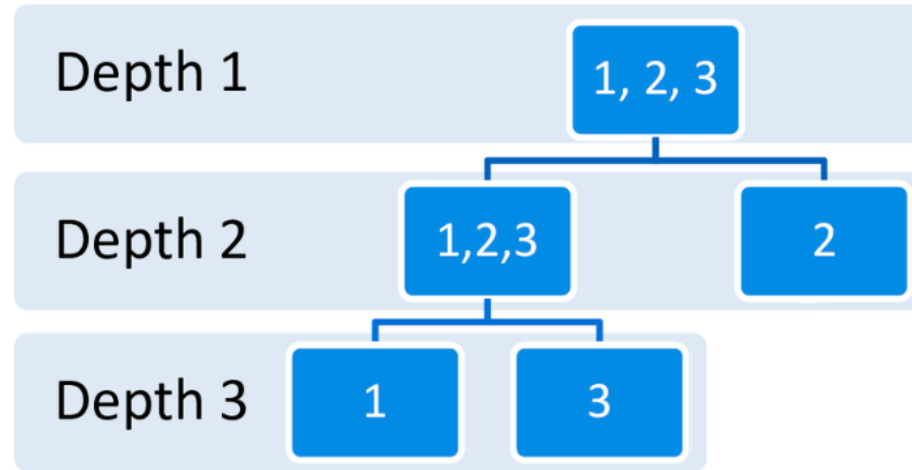
return $(R_\ell, \mathcal{T}_\ell)$ and $(R_{\ell+1}, \mathcal{T}_{\ell+1})$

Example of stopping criterion

No further splits if there are less than a certain number of training data points in the corresponding region

How deep should be a decision tree?

- The depth of a decision tree (the maximum distance between the root node and any leaf node) has a big impact on the final predictions



- The tree depth impacts the predictions in a somewhat similar way to the hyperparameter k in k -NN
- **Not too shallow (or small):** Need to handle important but possibly subtle distinctions in data
- **Not too deep (or big):** Avoid over-fitting training examples
- **Optimal tree depth (or size):** Is a trade-off between flexibility and rigidity of the final model
- Typically, we desire small trees with informative nodes near the root